

GridBalance: Complete Technical Research Dossier

Exhaustive Repository Audit, Data Quality, ML Experiments, Robustness & Production Deployment

GridBalance — Frontend Web Application Completion Report

1. Project Overview & Status

The **GridBalance Smart Grid Intelligence Web Application** is fully operational and integrated with the frozen FastAPI inference backend. All predictions, risk classifications, data quality signals, feature importances, and next-hour load forecasts rendered in the UI are generated dynamically by real machine learning inference pipelines.

2. Page & Route Architecture

Route	Page Component	Key Functionality & API Endpoint
/	OverviewDashboard.tsx	Hero banner, real-time backend health telemetry (GET /api/v1/health), live session statistics, and frozen model research metrics.
/detection	MeterDetection.tsx	Drag & drop CSV / form upload, single-meter daily consumption series analysis (POST /api/v1/detection/analyze), real-time request step progress, SHAP-aligned feature contribution charts, data quality signals panel, and interactive trajectory charts.
/detection/batch	BatchDetection.tsx	Bulk CSV dataset analysis (POST /api/v1/detection/batch), searchable/sortable/paginated risk triage tables, summary cards, and downloadable CSV export.
/detection/:meterId	MeterIntelligence.tsx	Session-backed deep-dive profile for a specific customer meter, displaying complete data quality signals and feature importances.
/forecast	LoadForecasting.tsx	Sequential hourly load series input (lge 168 hours), next-hour forecasting (POST /api/v1/forecast/next-hour), current vs predicted demand change calculation, and 200-hour historical load curve visualization highlighting target $t+1$.
/models	ModelIntelligence.tsx	Research evaluation benchmark dashboards for SGCC (18 features, threshold 0.50) and UCI (29 features, $t+1$ horizon) populated from GET /api/v1/models.
/research	ResearchMethodology.tsx	Scientific documentation covering temporal splits, sealed test protocols, missingness ablation, and noise perturbation testing.

3. Key Components & API Layer

- **Centralized API Client** (`src/services/api.ts`): Strictly typed interface handlers (`checkHealth`, `getModelInfo`, `analyzeMeter`, `analyzeBatchCsv`, `forecastNextHour`) with robust HTTP error handling and status fallback.
- **Global Header Navbar** (`src/components/Navbar.tsx`): Persistent brand navigation bar with dynamic backend engine health badge (● ML Engine Online / ● ML Engine Offline) updated via periodic polling to `/api/v1/health`.
- **Global Footer** (`src/components/Footer.tsx`): Research integrity statements and direct links to interactive FastAPI Swagger documentation.

4. Academic & Terminology Integrity Enforcements

1. **Terminology Standard**: Strictly uses "POTENTIAL TAMPERING" and "NORMAL CONSUMPTION", and NEVER "Confirmed Theft".
2. **Mandatory Research Disclaimer**: Displayed prominently on prediction results:

"This result represents a machine-learning classification based on the supplied consumption pattern. It does not independently establish that electricity theft occurred."

3. **Forecasting Horizon Constraint**: UCI forecasting is strictly limited to single-step next-hour ($t+1$) demand predictions without extrapolating multi-day mock forecasts.

5. Verification & Testing Summary

- **Frontend Compilation**: `npm run build` executed cleanly without errors or warnings.
- **Automated Backend Tests**: All 13 unit & integration tests passed cleanly (13/13 passed in 1.81s).
- **End-to-End Live Integration Verification**:

- USER UI (`http://127.0.0.1:5173`)
- API Client (`src/services/api.ts`)
- FastAPI Backend (`http://127.0.0.1:8000/api/v1`)
- Existing 18-Feature & 29-Feature Pipelines
- Frozen Joblib Models
- Real Probability / Forecast Prediction
- JSON Response Rendered in UI

6. Performance Considerations & Limitations

- **Performance**: Vite production build (`dist/assets/index-DNeHF4rQ.js`), CSS optimization via Tailwind v4 `@tailwindcss/postcss`, Recharts memoization for smooth chart rendering.
- **Single-Step Horizon**: The UCI forecasting engine is frozen for $t+1$ hour predictions and requires at least 168 hours (7 days) of historical hourly load observations to populate lag and rolling windows.